

UTRYON — AI Body Scan & Digital Fit Profile Application

Technical Documentation & Architecture Guide

Welcome to the comprehensive technical documentation for **UTRYON Digital Fitting Room**. This document is designed to explain **everything** about how this web application works — from high-level user workflows down to computer vision algorithms, backend provider architectures, state machines, and mathematical regression formulas.

You can use this guide to explain the application to non-technical stakeholders, product managers, or engineering teams.

1. Project Overview & Core Value Proposition

What is UTRYON?

UTRYON is a client-first, web-based AI body measurement application. It allows users to capture or upload two full-body photos (one **Front View** and one **Side View**) to automatically compute **13+ anatomical body circumference and length measurements** (such as chest, waist, hips, shoulder width, arm length, inseam, etc.) without requiring a physical measuring tape.

Core Goals & Features:

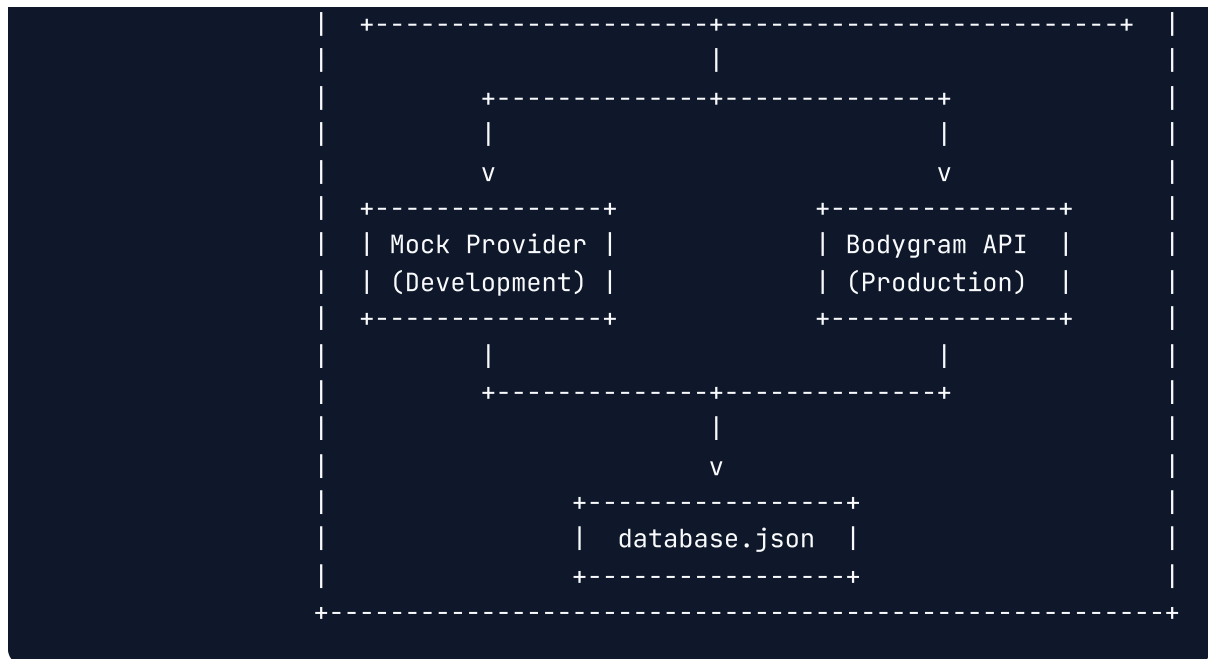
1. **Guided Real-Time Camera Scan:** Uses device cameras to track the user's posture live with a visual skeleton overlay and step-by-step voice guidance.
2. **5-Point Computer Vision Validation:** Checks person detection, full-body visibility, upright standing posture, exact 90° orientation turn, and 1.0-second posture stability before taking a photo.
3. **Photo Upload Mode:** Supports users without live camera access by allowing photo uploads, complete with automated computer-vision quality checks.

- ## 2. System Architecture Overview

```

+-----+
|                                     |
|                               BROWSER (SPA)                                |
|                                     |
| +-----+ +-----+               |
| |   index.html   | |       index.css        | |
| | (SPA Screen DOM) | | (Dark Glass Studio)    | |
| +-----+ +-----+               |
|           |                       |
|           v                       v
|
| +-----+
| |                               AppController (app.js)                      |
| |      (State Management, Router, Garment Ease)                          |
| +-----+
| |           |           |           |
| |           v           v           v
|
| +-----+ +-----+ +-----+
| | Camera | | Scan Quality | | Voice Engine |
| | Manager | | Engine     | | (Web Speech) |
| | (camera) | | (MediaPipe) | | (voice.js) |
| +-----+ +-----+ +-----+
|
+-----+-----+-----+
|           |           |
|           | (Base64 Photos) | (33 Pose Landmarks)
|           v           v
|
+-----+
|                                     |
|                               BACKEND SERVER                              |
|                               (server.js)                                |
|                                     |
| +-----+
| |                               Express REST API                           |
| |          (/api/scan/estimate, /api/scans)                             |
| |

```

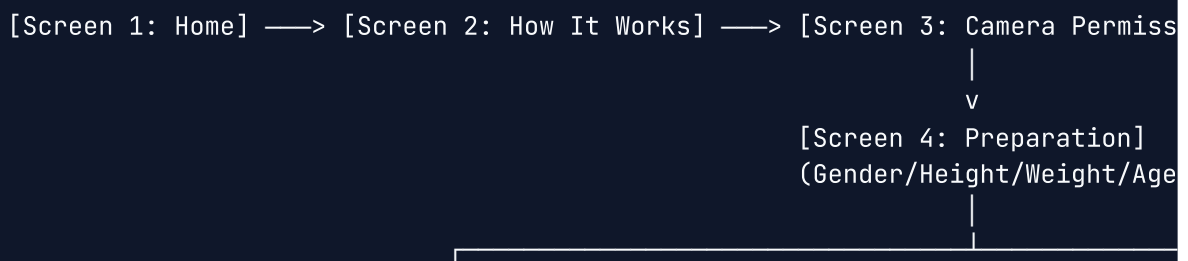


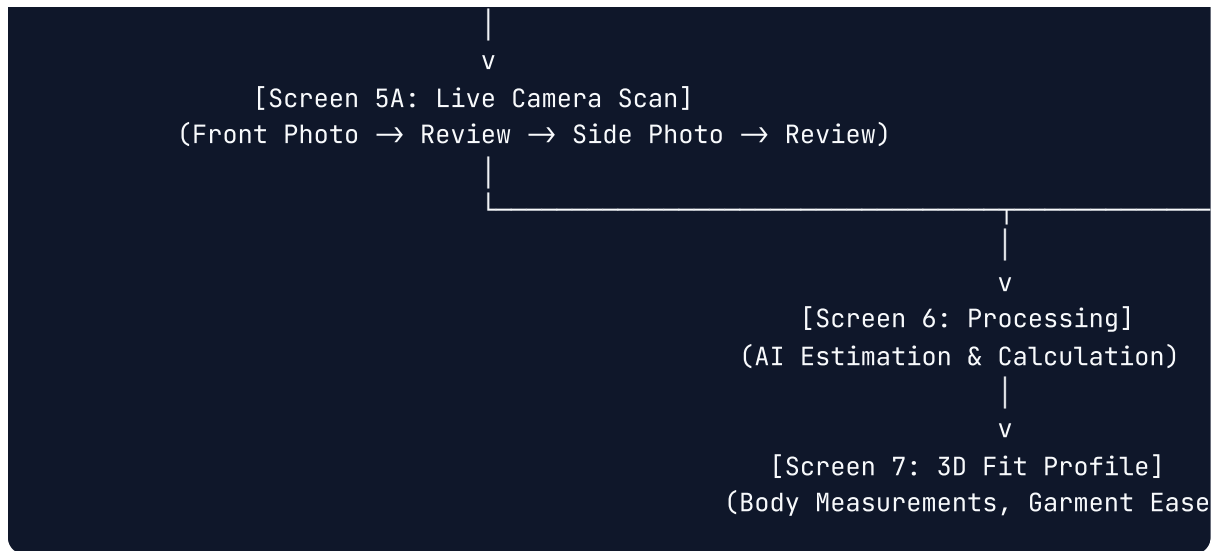
Technology Stack:

- **Frontend Core:** Vanilla JavaScript (ES6+ Classes), HTML5, Vanilla CSS3 (Custom Design System with Google Font 'Outfit').
- **Computer Vision:** Google MediaPipe Pose API (`@mediapipe/pose`) loaded via CDN for 33 3D landmark keypoint detection.
- **Audio & Voice:** Web Speech API (`SpeechSynthesis`) for text-to-speech guidance and Web Audio API (`AudioContext`) for synthetic beep sounds.
- **Backend Framework:** Node.js with Express.js (`server.js`).
- **Database:** Simple, persistent JSON file storage (`database.json`) handled with atomic read/write logic.

3. Step-by-Step User Flow & Screen Map

The application guides the user through 7 main screen states inside a single HTML page:





1. **screen-entry (Home)**: High-level overview presenting options to start camera scanning or upload existing photos.
2. **screen-how-it-works** : Explains the 3 simple steps: Position → Follow → Get Measured.
3. **screen-camera-permission** : Requests device camera permissions and offers retry / simulation mode fallbacks if camera access is denied.
4. **screen-preparation** : Crucial setup step where the user enters key physical metadata (Assigned Gender, Height in cm, Weight in kg, Age) required to calibrate baseline bone structure.
5. **screen-camera-scan (Live Mode)**: Full-screen interactive camera scanner with live skeleton overlay, positioning box, checklist HUD, status pills, and voice assistance.
6. **screen-upload-scan (Upload Mode)**: Multi-step upload workflow for users who prefer uploading existing photos.
7. **screen-processing** : Animated 3-stage progress loader ("Analyzing scan" → "Calculating measurements" → "Creating summary") while contacting backend API endpoints.
8. **screen-fit-profile (Dashboard)**: Interactive results screen with Metric/Imperial toggles, detailed body measurement breakdown, garment ease explorer, and persistent profile saving.

4. Deep Dive into Technical Modules

4.1 Computer Vision & Posture Engine (`public/js/scan-quality-engine.js`)

The pose engine uses **MediaPipe Pose**, which tracks 33 3D body keypoints (eyes, nose, shoulders, elbows, wrists, hips, knees, ankles, heels, foot index) in real-time at up to 60 FPS.

1. Temporal Landmark Smoothing

To prevent skeletal jitter on video feeds, the engine applies an **alpha-blended exponential moving average filter**: $\text{Smooth}_t = 0.70 \times \text{Raw}_t + 0.30 \times \text{Smooth}_{t-1}$

2. The 5-Point Quality Checklist:

Before allowing a photo capture, `ScanQualityEngine.evaluateFrame()` evaluates five strict criteria:

- Person Detection:** Checks if 33 landmark points are active and computes a tracking confidence percentage based on key joint visibility scores: $\text{Confidence} = \frac{\sum \text{Visibility}_i}{N} \times 100\%$
- Full Body Framing:** Verifies that the head ($y \leq 0.48$) and ankles/feet ($y \geq 0.52$) are completely within the frame boundaries.
- Upright Posture:**
 - Computes knee angles ($\angle \text{Hip-Knee-Ankle}$) and ensures legs are straight ($\geq 150^\circ$).
 - Computes torso tilt angle relative to vertical axis ($\leq 12^\circ$).
- Directional Orientation (Front vs. Side):**
 - **Front View:** Checks shoulder width relative to frame ratio (> 0.080).
 - **Side View:** Calculates estimated 3D body rotation angle using shoulder depth (Δz), hip depth, and shoulder width contraction: $\text{RotationAngle} = 0.60 \times \arctan\left(\frac{\Delta z_{\text{shoulder}}}{\Delta x_{\text{shoulder}}}\right) + 0.40 \times \arccos\left(\frac{\text{Width}}{\text{RefWidth}}\right)$

Triggers `TURN_SIDE` instruction if rotation angle $< 68^\circ$.

- Multi-Frame Stability:** Requires the user to hold still for **15 consecutive frames (~1.0 second)** under a jitter threshold ($\text{Jitter} < 0.016$).

4.2 Voice Guidance State Machine (`public/js/voice.js`)

To prevent overlapping audio feedback or prompt spamming, `VoiceEngine` implements a sequential state machine wrapping the browser's native `window.speechSynthesis` Web Speech API:

- **3.5-Second Cooldown:** Each unique instruction ID (e.g. `T00_CLOSE` , `TURN_SIDE`) has an independent cooldown timer before it can repeat.
- **Priority Tiering:** Instructions carry priority ranks (1 = highest, 8 = lowest). Higher priority alerts interrupt low-priority ones.
- **Movement Suppression:** Suppresses corrective verbal prompts while `isUserMoving = true` , giving the user quiet time to adjust position.

- **SpeakAsync(text) Promise Wrapper:** Allows the system to wait until an utterance finishes speaking before initiating countdown timers.

4.3 Live Camera Controller & Hardware Simulation (`public/js/camera.js`)

The `CameraManager` handles the lifecycle of live video streams and overlay rendering:

1. Camera Lifecycle State Machine:

IDLE → POSITIONING → CORRECT_POSITION → CAPTURE_NOTICE → COUNTDOWN → CAPTURING → CAPTURED

- During `POSITIONING`, the system continuously checks posture.
- Once ready, it locks pose detection, announces *"Perfect! You're in position"*, triggers a 3-2-1 countdown with synthesizer audio beeps (880Hz via `AudioContext`), flashes the canvas, and captures a high-resolution JPEG frame directly from the `<video>` element.

2. Photo Review Modal:

After capturing a frame, a modal opens to run a final quality verification on the image. If posture passes, the user clicks **Use Photo** to proceed to the Side scan.

3. Virtual Mock Camera Mode:

If camera hardware is unavailable or disabled, `startMockCamera()` generates an in-memory HTML5 Canvas stream rendered at 30 FPS. It draws an animated 2D studio scene with floor grid and a moving human figure, allowing full testing of the scan workflow without a webcam.

4.4 Backend Provider Architecture & APIs (`server.js`)

The backend server is structured with a **Provider Design Pattern**, allowing seamless switching between local development testing and production external APIs via environment variables (`BODYGRAM_PROVIDER_MODE`).

```
class BodyMeasurementProvider {
  async estimate({ frontPhoto, sidePhoto, gender, height, weight, age }) { ..
}
```

Mode A: Mock Provider (`MockMeasurementProvider`)

Uses statistical anthropometric regression heuristics derived from body proportions to generate realistic body measurements with a tiny random variation ($\pm 1.5\%$) for local testing:

- **Female Equations Sample:**
 - Chest = $0.46 \cdot \text{Height} + 0.18 \cdot \text{Weight}$
 - Waist = $0.38 \cdot \text{Height} + 0.16 \cdot \text{Weight}$
 - Hip = $0.50 \cdot \text{Height} + 0.22 \cdot \text{Weight}$
 - Inseam = $0.44 \cdot \text{Height}$
- **Male Equations Sample:**
 - Chest = $0.49 \cdot \text{Height} + 0.22 \cdot \text{Weight}$
 - Waist = $0.42 \cdot \text{Height} + 0.24 \cdot \text{Weight}$
 - Hip = $0.47 \cdot \text{Height} + 0.25 \cdot \text{Weight}$
 - Inseam = $0.45 \cdot \text{Height}$

Mode B: Production Provider (`BodygramMeasurementProvider`)

Communicates with the Bodygram REST API (<https://api.bodygram.com/v1/estimations>). Converts height to millimeters (cm $\times$ 10) and weight to grams (kg $\times$ 1000) as specified in official Bodygram API protocols.

REST API Endpoints:

Method	Endpoint	Description
POST	<code>/api/scan/estimate</code>	Accepts base64 <code>frontPhoto</code> , <code>sidePhoto</code> , and <code>metadata</code> (<code>gender</code> , <code>height</code> , <code>weight</code> , <code>age</code>). Runs active provider estimation and normalizes measurement units into centimeters (<code>cm</code>).
GET	<code>/api/scans</code>	Returns scan history array sorted newest first.

Method	Endpoint	Description
POST	/api/scans	Saves a completed fit profile scan into database.json .
GET	/api/scans/:id	Retrieves a specific scan profile by ID.

4.5 Resilient Client-Side Fallback & Garment Ease Explorer

(public/js/app.js)

AppController ties all user interface elements together:

1. Resilient Offline Fallback

If the network connection to /api/scan/estimate times out or fails, app.js catches the exception and executes an **in-browser client-side regression algorithm**. This guarantees that a user in the field never experiences a crash or broken screen state.

2. Unit System Formatting (Metric vs. Imperial)

Converts measurements dynamically on the fly: - **Centimeters to Inches:** $\text{inches} = \frac{\text{cm}}{2.54}$ - **Height Display:** Formats centimeters into feet and inches (e.g. 170 cm $\rightarrow$ 5'7").

3. Garment Ease Explorer

Calculates garment drafting specs based on user measurements and clothing category comfort allowances (ease):

Garment Category	Ease Formula	Target Garment Feature
Shirt	Chest + 5.0 cm, Waist + 5.0 cm, Sleeve = ArmLength + 2.0 cm	Standard relaxed comfort fit
Jacket	Chest + 9.0 cm, Waist + 9.0 cm, Sleeve = ArmLength + 3.5 cm	Outerwear layering room

Garment Category	Ease Formula	Target Garment Feature
Trousers	Seat = Hip + 4.0 cm, Waist = Waist + 2.0 cm, Inseam = Inseam	Seating room & waist movement

5. Summary Schema Definitions

Normalized Fit Profile JSON Schema:

```
{
  "id": "scan_3ed62d3f7635305e",
  "timestamp": "2026-08-30T21:02:37.703Z",
  "provider": "mock",
  "confidence": "Excellent",
  "metadata": {
    "gender": "female",
    "height": 170.0,
    "weight": 65.0,
    "age": 25
  },
  "measurements": {
    "height": 170.0,
    "chest": 91.0,
    "waist": 74.9,
    "hip": 99.1,
    "shoulderWidth": 37.3,
    "neck": 32.3,
    "upperArm": 24.9,
    "armLength": 54.2,
    "wrist": 15.4,
    "thigh": 18.0,
    "inseam": 75.1,
    "outseam": 101.6,
    "torsoLength": 60.6
  },
  "estimationToken": "token_mock_d8ffec96f133292b"
}
```

6. How to Run & Test the Application

1. Installation & Local Server Setup:

```
# Install dependencies
npm install

# Start development server with auto-reload
npm run dev

# Access in browser:
http://localhost:3000
```

2. Environment Variables (`.env` file):

```
PORT=3000

# Provider configuration: 'mock' (default) or 'production'
BODYGRAM_PROVIDER_MODE=mock

# Bodygram API credentials (Required only in production mode)
BODYGRAM_API_KEY=your_api_key_here
BODYGRAM_CLIENT_ID=your_client_id_here
```

3. Simulation Testing (Without Camera Hardware):

Append `?mock=true` or `?mockCamera=true` to the URL in your browser:

```
http://localhost:3000/?mock=true
```

This activates the virtual camera stream and injects synthetic body pose landmarks to test the full scanning pipeline automatically.

7. How to Explain This Project to Others

The 1-Minute Executive Summary (For Non-Technical Audiences)

"UTRYON is a digital fitting room application that acts like a smart tailor inside your web browser. Instead of using a physical measuring tape, you stand in front of your phone or laptop camera and take two quick guided photos — front and side. The app uses AI to check your posture in real time, guides you with voice prompts, and extracts 13+ detailed body measurements (like chest, waist,

hips, and sleeve length) so you can find perfect-fitting clothes when shopping online."

The 3-Minute Technical Summary (For Developers / Engineers)

"UTRYON is an end-to-end Single Page Application built with Vanilla JS on the frontend and an Express Node server on the backend. The core client engine integrates Google MediaPipe Pose to track 33 3D landmark keypoints live on camera feeds. It enforces a 5-point quality checklist covering person detection, framing boundaries, posture uprightness, trigonometric 90° body orientation, and temporal stability.

Once quality criteria are met, it captures high-resolution frames, normalizes inputs, and sends them to a backend provider architecture that interfaces with the Bodygram REST API (or an offline statistical regression engine in development mode). If network connectivity drops, the frontend transparently executes an offline regression fallback algorithm, rendering an interactive 3D fit profile dashboard complete with garment ease calculations for custom clothing construction."

Created for UTRYON Digital Fitting Room Project Documentation.